

Warm up

What is the value of `result` after executing the following snippet?

```
#include <stdio.h>
#include <limits.h>

int main(void) {
    int x = INT_MAX;
    unsigned int y = UINT_MAX, z = 1;
    int result = (x + z > 0) + (y + z > 0);
    return 0;
}
```

Warm up

What is the value of `result` after executing the following snippet?

```
#include <stdio.h>
#include <limits.h>

int main(void) {
    int x = INT_MAX;
    unsigned int y = UINT_MAX, z = 1;
    int result = (x + z > 0) + (y + z > 0);
    return 0;
}
```

Answer: `result = 1`

- `z` is unsigned, so `x + z` is evaluated in unsigned arithmetic. `INT_MAX + 1` is 2^{31} (assuming 32-bit ints), so `x + z > 0` is true (1)
- `y + z` is also evaluated in unsigned arithmetic. `UINT_MAX + 1` is 2^{32} , which wraps around to 0, so `y + z > 0` is false (0)

Precept 5: Building and Debugging C Programs

Polly Ren

COS 217: Introduction to Programming Systems
Spring 2026
Princeton University

February 10, 2026

Overview

1 Building multi-file programs

2 Debugging in gdb

3 Summary

Recall: the four-step build process, again

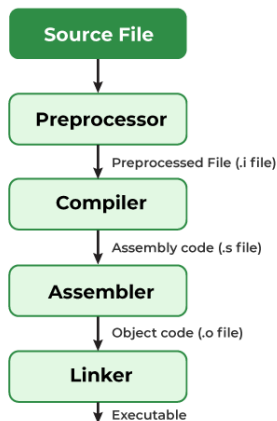


Image source:
<https://www.geeksforgeeks.org/c/compiling-a-c-program-behind-the-scenes/>

- Preprocessing
 - Removes comments and handles preprocessor directives
 - `gcc217 -E hello.c > hello.i`
- Compiling
 - Translates preprocessed C code into assembly code
 - `gcc217 -S hello.i`
- Assembling
 - Translates assembly code into machine code (object file)
 - `gcc217 -c hello.s`
- Linking
 - Combines object files and libraries into an executable
 - `gcc217 hello.o -o hello`
- Combined: `gcc217 hello.c -o hello`

Multi-file programs

- Large programs are often split across multiple source files
- Each source file is compiled into an object file separately
- The object files are then linked together to create the final executable
- Advantages:
 - Easier to manage and organise code
 - Only need to recompile (preprocess, compile, assemble) changed files
 - Facilitates code reuse

Writing multi-file programs

Walkthrough of `intmath4` program, but here are the main points:

- static functions have internal linkage (only use in same `.c` file)
 - Similar to `private` in Java
- *Forward declarations* allow functions to be used before definition
 - Provide function signature (return type, name, parameter types, parameter names optional)
 - Why: compiler needs to see function signature (at minimum) before function call
- Header files (`.h`) provide the “interface” for a module
 - Function headers (declarations) go here
 - Implementation (definitions) should be in `.c` files
 - `#include` directive to include header files
 - ◇ Use quotes (`"intmath.h"`) for user-defined headers
 - ◇ Use angle brackets (`<stdio.h>`) for system headers

Building multi-file programs in steps

- Preprocess each source (.c) file into an intermediate file (.i)
 - `gcc217 -E intmath.c > intmath.i`
 - `gcc217 -E testintmath.c > testintmath.i`
- Compile each .i file into an assembly file (.s)
 - `gcc217 -S intmath.i`
 - `gcc217 -S testintmath.i`
- Assemble each .s file into an object file (.o)
 - `gcc217 -c intmath.s`
 - `gcc217 -c testintmath.s`
- Link the object files together to create a single, final executable
 - `gcc217 intmath.o testintmath.o -o intmath`
- Advantages:
 - If only one source file changes, only need to recompile that file and relink the object files

A shortcut for building multi-file programs

- One command: `gcc217 intmath.c testintmath.c -o intmath`
 - Must specify as command-line arguments each `.c` file that comprises the program
 - Must not specify as command-line arguments any `.h` file, these are handled by preprocessor through `#include`
 - Across all `.c` files, there must be exactly one `main` function
 - ◇ Why? `main` is the entry point of the program, so there can only be one
 - ◇ Which stage of the build process will complain if there are multiple `main` functions (in different `.c` files)? Linking stage

```
/usr/bin/ld: intmath.o: in function 'main':
intmath.c:(.text+0x0): multiple definition of 'main';
testintmath.o:testintmath.c:(.text+0x0): first defined here
collect2: error: ld returned 1 exit status
```

Preventing multiple inclusions

- Header files can be included in multiple source files
 - At best: inefficient because redundant declarations
 - At worst: compile-time errors due to multiple definitions
- To prevent multiple inclusions of the same header, use include guards

intmath.h:

```
#ifndef INTMATH_H          /* checks if INTMATH_H is not defined */
#define INTMATH_H          /* defines INTMATH_H */
...
#endif                    /* marks the end of the conditional */
```

- First time the header file is included, INTMATH_H is not defined, so the contents of the file are included and INTMATH_H is defined
- Subsequent inclusions of the header file will find that INTMATH_H is already defined, so the body of the file is skipped

Overview

1 Building multi-file programs

2 Debugging in gdb

3 Summary

What is gdb?

- gdb is the GNU Project Debugger
- Allows you to:
 - Step through code line by line
 - Inspect variable values and memory contents
 - Set breakpoints to pause execution at specific points
 - Analyse the state of a program at the time of a crash

Using gdb

- Compile your program with debugging information: `gcc217 -g prog.c -o prog`
- Start gdb with your executable: `gdb prog`
- Use `run` command to start the program within gdb
- Set breakpoints with `break (b)` command, e.g., `break main` or `break file.c:42`
- Step into code with `step (s)` and step over code with `next (n)`
- Inspect variables with `print (p)` command, e.g., `print x`
- Continue execution with `continue (c)` command

Useful gdb commands

What to do	Shorthand	What it does
run [<arg>...]	r [<arg>...]	run program with command line arguments
break <loc>	b <loc>	set a breakpoint
step	s	next instruction, step into function
next	n	next instruction, step over function
jump <loc>	j <loc>	jump to a location
continue	c	continue until next breakpoint or end
until <loc>	u <loc>	continue until location (function / line)
finish	fin	step out of current function
print <exp>	p	print value of expression
info breakpoints	i b	list all breakpoints
info locals	i locals	list local variables and their values
backtrace	bt	show call stack

More commands in *GDB Tutorial and Reference* handout

Overview

1 Building multi-file programs

2 Debugging in gdb

3 Summary

Summary

- Building multi-file C programs
- Debugging with gdb
- Dates to remember:
 - Assignment 1: due Tue, 2/10 at 9pm (tonight!)
 - Assignment 1 Quiz: Wed, 2/11 in lecture (tomorrow!)